

DSCC2019-9237

ON RECEDING HORIZON CHANCE CONSTRAINT MOTION PLANNING FOR UNCERTAIN MULTI-AGENT SYSTEMS

Yash Bagla

Department of Mechanical Engineering
Michigan State University
East Lansing, Michigan, 48824
Email: baglayas@msu.edu
yashbagla321@gmail.com

Vaibhav Srivastava

Department of Electrical and Computer Engineering
Michigan State University
East Lansing, Michigan, 48824
Email: vaibhav@egr.msu.edu

ABSTRACT

We propose and study a motion planning algorithm for multi-agent autonomous systems to navigate through uncertain and dynamic environments. We use a receding horizon chance constraint framework that allows for tuning the trade-off between the risk of collision and the infeasibility of paths. We consider sampling-based incremental planning algorithms and extend them to the case of multiple agents and dynamic and uncertain environments. The receding horizon control framework is used to incorporate sensor measurements at a fixed interval of time to reduce uncertainty about agents' state and environment. Our presentation focuses on rapidly-exploring random trees (RRTs) and the assumption of Gaussian noise in the uncertainty model. Our algorithm is illustrated using several examples.

1 Introduction

Multi-agent motion planning under motion and environment uncertainty is fundamental to safe and resilient operations of connected and autonomous vehicles. Efficient algorithms for these problems need to address several competing objectives, including, computational efficiency, (dynamic) path feasibility, and conflict resolution. In this work, we develop a novel incremental sampling based algorithm for multi-agent motion planning that addresses these challenges.

Sampling-based motion planning algorithms have been

studied extensively [1–3]; see [4, 5] for a survey. These algorithms are known to be versatile in terms of handling complex environments and different motion models. Recent developments have extended these algorithms to provide optimality guarantees [6]. Furthermore, these algorithms have been applied to challenging scenarios such as urban driving [7, 8].

Motion planning under uncertainty has also received significant attention. Belief space planning has been a prominent framework to address the uncertainty [9–13]. An alternative formulation involves chance-constrained planning [14–16]. In this paper, we adopt the latter framework and study the multi-agent planning problem.

The multi-agent planning using sampling-based methods has received some attention [17–20]. A key feature of these works is the decentralized implementation of the sampling based algorithms. Priority ordering of the robots is one of the popular techniques used in these decentralized implementations. A review of priority ordering techniques is presented in [21]. Auction algorithms [22, 23] are a popular class of algorithms used for priority ordering the robots. In these algorithms, an auction parameter which calculates the benefit in changing the path at each planning instant is used to decide the priority order. Recently, multi-agent motion planning under uncertainty has been studied in the belief-space planning framework [24].

Our work develops a framework for planning dynamically feasible paths in an uncertain and complex environment for a group of autonomous robots travelling from (distinct) initial po-

sitions to (distinct) goal positions. These paths must be probabilistically safe, i.e., the probability of collision with other agents and the obstacles present in the surroundings should be smaller than a desired constant at each time-step. To this end, we extend the Chance-Constrained RRT algorithm [14] to a multi-agent setting using the framework of [17]. We implement our algorithm using a receding horizon fashion in which each agent updates the estimates of its position at each time based on its sensory input and refines its path by exploring an alternative path with a smaller associated cost (e.g., fuel consumption or time taken to reach the goal). We also incorporate an obstacle avoidance check which makes the algorithm robust to abruptly appearing obstacles.

The remainder of the paper is organized in the following way. In Section 2, we present some background on relevant algorithms. Our problem setup is described in Section 3. In Section 4, we present the receding horizon priority-based multi-agent chance-constrained RRT algorithm, and illustrate it in Section 5. Finally, we conclude in Section 6.

2 Background

In this section, we review some algorithms that we build upon in this paper.

2.1 Rapidly Exploring Random Trees (RRT) Algorithm

The RRT algorithm was developed by LaValle and Kuffner [3]. It iteratively grows a tree with its root node located at the starting position of the robot. At each iteration, a sample point is drawn randomly in the feasible region of the map, and this point is then connected to the nearest node in the tree via a straight line. A new node is selected along this line at a fixed small distance from the nearest node. If the edge connecting the new node and the nearest node is collision free, then the new node and the edge are added to the tree and the nearest node is labeled as the parent node of the new node; otherwise the new node is discarded. An illustration of the tree expansion for RRT is shown in Figure 1. When we have a collision-free edge from the new node to the goal node (goal position), the new node is labeled as the parent of the goal node. The final path is given by the goal node as the last node and then tracing the parent of each node back to the start node.

2.2 Chance Constrained RRT

The Chance constrained RRT or CC-RRT algorithm extends the traditional RRT algorithm by incorporating motion and environmental uncertainties into the planning problem [14]. The CC-RRT generates a tree in which the new node is generated similar to the RRT. The new node and the edge to the nearest node are added to the tree if the probability of collision for this edge is less than a given upper bound. The algorithm assumes that the

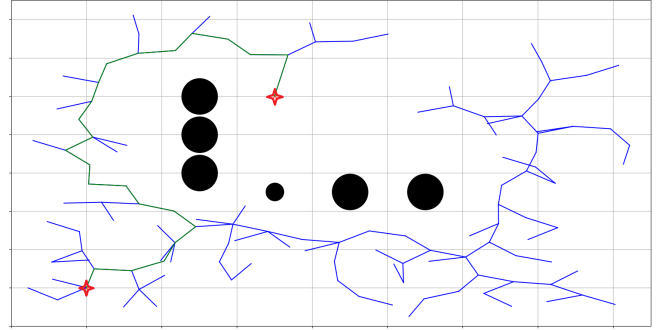


Figure 1. Growing a RRT: blue lines represent the RRT, green lines represent the final path, start and goal locations are represented by red star

edges are short enough so that the probability of collision for an edge is approximated well by the probability of collision when the robot is at the node at which the edge ends. The probability of collision at each node is calculated using a Gaussian noise model in which the variance in the position of the robot increases linearly with the length of the path to the associated node from the root node. An illustration of the tree expansion for the CC-RRT is shown in Figure 2. In this paper, we modify the CC-RRT algorithm and implement it in a receding horizon fashion so that we can incorporate the sensory data to improve the estimate of robots' position at each time.

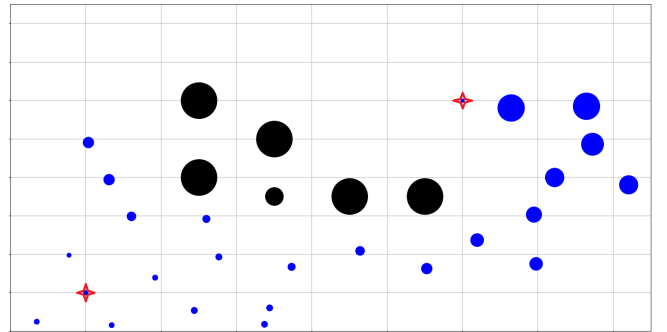


Figure 2. Growing a CC-RRT: blue discs represent the confidence sets for robot positions (they are centered at the mean robot position and have radius proportion to the standard deviation); black discs are static obstacles; and start and goal locations are represented by red star.

3 Problem Setup

In this section, we introduce our problem, describe modeling assumptions, and the receding-horizon chance constrained framework that we adopt in this paper.

3.1 Problem Statement

Consider a group of $b \in \mathbb{N}$ robots $B_1, B_2, \dots, B_b$ moving in a planar environment comprised of $s \in \mathbb{N}$ static obstacles $S_1, S_2, \dots, S_s$ with fixed, known locations, and $d \in \mathbb{N}$ dynamic obstacles $D_1, D_2, \dots, D_d$ with known stochastic trajectories, i.e., their motion is driven by known stochastic processes. The information about the trajectories of these obstacles might be available from some inference engine that has observed the behavior of these obstacles and provides a prediction of their trajectories with associated uncertainties. The robot motion is assumed to be uncertain.

Our objective is to navigate every robot from its initial position to a goal position such that its total probability of collision at each time step is less than a given constant M . The total probability of collision is the sum of the probabilities of collision of the robot with all the static and dynamic obstacles, and all other robots. Let $\mathbb{P}_{\text{collision}}^{B_j}(O, t)$ be the probability of collision of robot B_j with an object O at time t . The object O can be another robot B_i , $i \neq j$, a static object S_i , $i \in \{1, \dots, s\}$, or a dynamic obstacle D_i , $i \in \{1, \dots, d\}$. Then, the total probability of collision for robot B_j at time t is

$$\begin{aligned} \mathbb{P}_{\text{collision}}^{B_j}(t) = & \sum_{i=1}^s \mathbb{P}_{\text{collision}}^{B_j}(S_i, t) \\ & + \sum_{i=1}^d \mathbb{P}_{\text{collision}}^{B_j}(D_i, t) \\ & + \sum_{i=1, i \neq j}^b \mathbb{P}_{\text{collision}}^{B_j}(B_i, t). \end{aligned} \quad (1)$$

3.2 Sensing and Motion Model

The robots are assumed to be point masses with the following linear uncertain dynamics:

$$\begin{aligned} x_{t+1} &= Ax_t + Bu_t + \omega_t \\ x_0 &\sim \mathcal{N}(\hat{x}_0, P_{x_0}) \\ \omega_t &\sim \mathcal{N}(0, P_\omega), \end{aligned} \quad (2)$$

where $A \in \mathbb{R}^{2 \times 2}$ is the system matrix, $B \in \mathbb{R}^{2 \times n_u}$ is the input matrix, $u_t \in \mathbb{R}^{n_u} \setminus \{0\}$ is the input vector, and $\omega_t \in \mathbb{R}^2$ is the disturbance vector. We assume that x_0 is a Gaussian random vector with mean $\hat{x}_0$ and covariance P_{x_0} , and $\{\omega_t\}_{t \in \mathbb{N}}$ is an i.i.d. sequence of zero mean Gaussian random vectors with covariance P_ω . We assume that $\{\omega_t\}_{t \in \mathbb{N}}$ is independent of x_0 .

In the following, we assume that the robot model (2) is controllable and there exists some input u_t that, in expectation, moves the robot by a predefined distance in a desired direction in a single time step. We will refer to this predefined distance as the *step-size*. To avoid high computational complexity in obstacle

avoidance, we have assumed that the step-size is much smaller than the dimensions of the obstacles. This assumption ensures that it suffices to check for collisions at robot positions at discretized time instants in (2) and alleviates the computational cost of checking for collision within these time instants.

If A and B are identity matrices, the above dynamics reduces to a noisy integrator dynamics

$$x_{t+1} = x_t + u_t + \omega_t, \quad (3)$$

which are the dynamics of a noisy velocity-driven point mass. In this case, the covariance of the predicted position of the robot increases linearly with the prediction time.

Suppose that the robot can measure its position at each discretized time instant, e.g., using a GPS, and can update the estimate of its current location. Let the measurement $z_t \in \mathbb{R}^{n_z}$ of its own location by the robot at time t be

$$\begin{aligned} z_t &= C_t x_t + \delta_t \\ \delta_t &\sim \mathcal{N}(0, Q), \end{aligned} \quad (4)$$

where $C_t \in \mathbb{R}^{n_z \times 2}$ is the measurement matrix, and $\{\delta_t\}_{t \in \mathbb{N}}$ is an i.i.d. sequence of zero mean Gaussian random vectors with covariance Q and it is independent of $\{\omega_t\}_{t \in \mathbb{N}}$ and x_0 .

Let x_t^τ denote the random vector x_τ conditioned on the measurements $\{z_1, \dots, z_t\}$ and inputs $\{u_1, \dots, u_{\tau-1}\}$, where $\tau \geq t$. Let $\hat{x}_t^\tau$ and $P_{x_t^\tau}^\tau$ be the associated mean and covariance. For dynamics (2) and measurement model (4), x_t^τ is a Gaussian random vector and the following dynamics hold [25]

$$\begin{aligned} \hat{x}_{t+j}^\tau &= A \hat{x}_{t+j-1}^\tau + B u_{t+j-1} \\ P_{x_{t+j}^\tau}^\tau &= A P_{x_{t+j-1}^\tau}^\tau A^T + P_\omega, \quad j \in \mathbb{N}. \end{aligned} \quad (5)$$

Given the conditional mean and covariance of random variable x_t^τ , dynamics (5) yields the mean and covariance of x_{t+j}^τ for given future inputs. We will use these dynamics to predict future uncertainty in robots' positions, which will be used to determine collision probabilities.

Given the mean and covariance of x_t^τ , measurement z_{t+1} , and input u_t , the mean and covariance of $x_{t+1}^{\tau+1}$ can be computed using standard Kalman filter updates [25]:

$$\begin{aligned} \hat{x}_{t+1}^\tau &= A \hat{x}_t^\tau + B u_t \\ P_{x_{t+1}^\tau}^\tau &= A P_{x_t^\tau}^\tau A^T + P_\omega \\ \hat{x}_{t+1}^{\tau+1} &= \hat{x}_{t+1}^\tau + K_{t+1}(z_{t+1} - C_{t+1} \hat{x}_{t+1}^\tau) \\ P_{x_{t+1}^{\tau+1}}^{\tau+1} &= (I - K_{t+1} C_{t+1}) P_{x_{t+1}^\tau}^\tau, \end{aligned} \quad (6)$$

where K_{t+1} is the Kalman gain given by:

$$K_{t+1} = P'_{x_{t+1}} C_{t+1}^T (C_{t+1} P'_{x_{t+1}} C_{t+1}^T + Q)^{-1}. \quad (7)$$

In the following, we will use (6) in our receding horizon implementation to improve the estimates of the current robot positions and subsequently use them to reduce the uncertainty in the future estimated positions as determined using (5).

While the discussion in this paper is based on a linear stochastic model for robot motion, the ideas can be easily applied to nonlinear motion model with equation (5) replaced by approximate difference equations for the first and second moments of the state of the nonlinear model, and the Kalman filter (6) replaced with an appropriate nonlinear filter [25]. Furthermore, the measurement model may not be limited to GPS measurements but may include a cooperative estimate of the position of the robot determined, e.g., in the spirit of [26], using GPS measurements, landmark measurements, and measurements of the robot position relative to other robots.

At time t , let $f_O^t(\tau, x)$ be the probability density function at time $\tau \geq t$ of the position of object $O \in \{B_1, \dots, B_b, D_1, \dots, D_d\}$. For $O \in \{B_1, \dots, B_b\}$, $f_O^t(\tau, x)$ can be computed using (5), and we assume that $f_O^t(\tau, x)$ can be computed using some inference engine for $O \in \{D_1, \dots, D_d\}$. In the following, we assume that each robot has access to the latter densities. We also assume that robots are priority ordered and the robots with lower priority have access to $f_B^t(\tau, x)$ for each robot B with higher priority. Note that under the Gaussian noise assumption, the knowledge of probability density function reduces to the knowledge of the mean vector and the covariance matrix.

For robot B_j , the probability of collision with the static obstacles S_i at time $\tau \geq t$ predicted at time t is the probability that a random variable with density $f_{B_j}^t(\tau, x)$ belongs to the set S_i . While robots/dynamic obstacles are point masses, we take a robust measure of their positions captured by the associated α -confidence set, where α is a tunable parameter. In particular, at time t , the predicted position of each robot/dynamic obstacle O at current/future time $\tau \geq t$ is replaced with the α -confidence set for $f_O^t(\tau, x)$. Subsequently, we compute the collision probability of robot B_j with a dynamic obstacle or another robot O by computing the probability that the robot B_j is contained in the α -confident set of O . For the purposes of our simulations, we choose $\alpha = 0.99$. The above collision probabilities can be computed in closed form in certain scenarios (see e.g., [27]); however, in our simulations we adopt a simpler Monte-Carlo strategy for approximately computing these probabilities. In particular, we sample 1000 points from the (predicted) distribution of the robot position and determine the fraction of points contained in the static obstacles or in the α -confident sets for the positions of other robots and dynamic obstacles.

3.3 Receding Horizon Chance Constrained Framework

Receding horizon control [28] is a general purpose control scheme for uncertain dynamical systems that (i) updates the system state at each time using the received measurements, and then (ii) selects the control action at that time by solving a constrained optimization problem using predicted system evolution. The above loop of state update and (re)planning ensures that the control inputs are refined as new observations are received. The receding horizon frameworks provides an effective online controller for complex uncertain systems. In the chance-constrained framework [16], environmental constraints on the stochastic system trajectories are imposed to hold with high probability at each time instant. Such formulation provides a probabilistic feasibility to the trajectories and is particularly powerful when the noise model has a large support and the worst case noise (which occurs with very small probability) yields no feasible solution.

In this paper, we adopt the receding horizon chance constraint framework and apply it to the multi-agent chance-constrained motion planning. The uncertainty in our system model is mainly because of four major factors: the uncertainty in the initial positions of the robots, the uncertainty in the robot motion model, the uncertainty in the sensing model, and the uncertainty in the environment due to uncertain moving obstacles.

In our implementation, the Kalman filter updates (6) are used to update the estimate of the robots' position at each time. Furthermore, the estimated positions and planned trajectories of the other robots are also updated at each time. This information is used in a receding horizon fashion to re-plan a superior trajectory.

4 Multi-Agent Planning using Receding Horizon CC-RRT

We now present the multi-agent receding horizon chance constrained RRT algorithm. While the presentation here builds upon the RRT algorithm, the proposed algorithm is general enough to incorporate any other incremental sampling based planner, such as RRT* [6], CL-RRT [7], and Dubins-RRT [29]. We first present the priority-based multi-agent CC-RRT in Algorithm 1. We then use this algorithm in a receding horizon fashion to develop the receding horizon priority-based multi-agent CC-RRT presented in Algorithm 2. Both these algorithms assume that the robots are priority ordered, and the robots can broadcast their planned trajectories, i.e., the time sequence of their expected positions and the associated covariance.

4.1 Priority-based Multi-agent CC-RRT

The priority-based multi-agent CC-RRT is summarized in Algorithm 1. Every robot starts at its (uncertain) initial position, where uncertainty is due to noise in the sensory information, e.g.

Algorithm 1: Priority-based Multi-agent CC-RRT

Result: Individual Paths for each robotInitialize with t , $\Lambda_i = []$;**for** $i = 1$ to b **do**

Grow CC-RRT [14], choose the best path $\Lambda_i^*(t)$ such that estimated $\mathbb{P}_{\text{collision}}^{B_i}(\tau) < M$, for each $\tau \geq t$;
Broadcast trajectory i.e. sequence of nodes with covariance and mean;
 $\Lambda_i(t) \leftarrow \Lambda_i^*(t)$;

end

Algorithm 2: Receding Horizon Priority-based Multi-agent CC-RRT

Input: Initial paths, $\Lambda_i(0)$, for each robot B_i ;**Result:** Closed-loop modifications over time

Find initial path for each agent using Algorithm 1;

while *Goal is not reached* **do****for** $i = 1$ to b **do**

Navigate to next node;
Add starting $\Lambda_i(t)[0]$ to path taken;
Get sensory measurement;
Update the root node;
 $\Lambda_i(t)[0] \leftarrow \Lambda_i(t)[1]$;
Perform a Lazy check for path feasibility;
Grow CC-RRT [14], choose the best path $\Lambda_i^*(t)$ such that estimated $\mathbb{P}_{\text{collision}}^{B_i}(\tau) < M$, for each $\tau \geq t$;
if $\text{Cost}(\Lambda_{\text{new}}^*) < \text{Cost}(\Lambda_i(t))$ or *Lazy check gives collision* **then**
 $\Lambda_i(t) \leftarrow \Lambda_{\text{new}}^*$;
 Broadcast trajectory i.e. sequence of nodes with covariance and mean;

else

Navigate on the same path;

end**end****end**

the GPS signal. Algorithm starts with a pre-decided priority order and the robots are activated (plan their trajectory) in that order. The active robot grows a CC-RRT such that the probabilistic collision constraints on each edge are satisfied. The mean and covariance update required to calculate these collision probabilities are made using equation (5) as discussed in Section 3.2.

An edge is classified safe with respect to other robots and moving obstacles, if the parent and child nodes associated with the edge are safe, i.e., the probability of collision at these nodes is less than the desired value M and the edge does not intersect

with the path of the other robots and obstacles.

Since the robots plan their paths in a pre-defined sequence, the latter robots are aware of the trajectory of the previous robots in the sequence and can incorporate them in their collision probability calculations. Once each robot finds a path such that each edge on its path is safe, the motion plan is activated.

4.2 Receding Horizon Priority-based Multi-agent CC-RRT

We implement the priority-based multi-agent CC-RRT algorithm in a receding horizon fashion and the resulting algorithm is presented in Algorithm 2. Once each robot plans an initial trajectory, it broadcasts the trajectory information: nodes (expected positions) on the trajectory, the time to reach them, and covariances associated with the positions at nodes. As each robot traverses their trajectory, after visiting a pre-determined number of nodes, it updates its state based on received observations. Then, each robot, in the order of its priority, plans a new trajectory using Algorithm 1 from a subsequent pre-determined node in its current trajectory. If the new trajectory is superior than the current trajectory based on some cost measure such as length, probability of collision, or energy, then the robot updates the trajectory and broadcasts this information to every other robot. This gives us the flexibility to update the estimate of the free space, improve efficiency and re-plan if a new obstacle is introduced.

For small values of M , the estimated free space may be too narrow and sampling based incremental planner may require a lot of sampled points to find the trajectory. The receding horizon framework, helps reduce the uncertainty in the agent and environment estimates which in turn leads to a relatively bigger estimated free space.

4.2.1 Lazy Check for Path feasibility While the trajectories are designed such that the probability of collision is smaller than M on each edge, these probabilities can cascade and lead to a collision. In particular, consider a path Λ comprised of E nodes and let $\mathbb{P}_1, \mathbb{P}_2, \dots, \mathbb{P}_E$ be the probabilities of collision at each node. Then the probability of safely navigating on a path Λ is given by:

$$\mathbb{P}_{\text{safe}}(\Lambda) = \prod_{i=1}^E (1 - \mathbb{P}_i). \quad (8)$$

If the probability of collision at each node is lower bounded by a constant $\mu > 0$, then $\mathbb{P}_{\text{safe}}(\Lambda) \leq \mu^E$. Equivalently, the probability of collision $\mathbb{P}_{\text{collision}}(\Lambda) \geq 1 - \mu^E$, which grows exponentially to 1 as a function of the number of nodes. Furthermore, if a new obstacle is introduced in the environment, the current trajectory may become infeasible. In the receding horizon planning, as discussed earlier, a new trajectory is designed and compared against

the current trajectory using some cost function. If this cost function is the probability of collision, then the new trajectory will be preferred in the event of an (almost) impending collision. However, if an alternative cost metric is selected, then the new trajectory may be discarded even if it is substantially safer to navigate.

The lazy check routine alleviates this problem by recalculating the probability of collision for the current trajectory over next few nodes and selects the new trajectory if the current trajectory is not sufficiently safe. This check is called a lazy check because it only checks for a collision in immediate future of the robot given the trajectories of all the other agents.

Figure 3 illustrates a scenario in which the current and goal locations of the robots are selected such that the direct path connecting these locations will lead to a collision. Such current positions may result from a cascade of improbable events in the trajectories. The lazy check algorithm detects collision for these trajectories and forces the robots to move around each other and take longer paths. The overall execution of the algorithm is summarized in Figure 4.

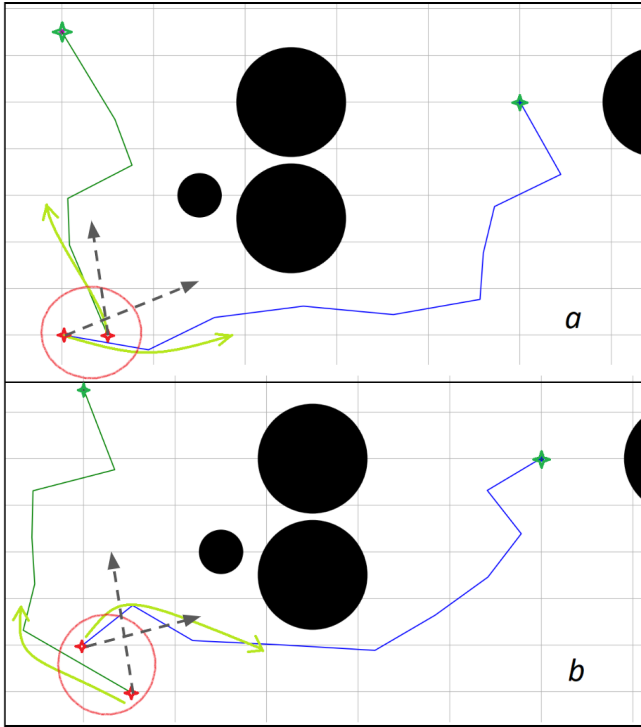


Figure 3. Lazy check: in both cases *a* and *b* the robots find a path around other agent's path if there is a possibility of collision. The grey dashed arrows represent the direct path towards the goal which is avoided by the robots and they take a longer path (along green arrows) around the other robots to avoid collision

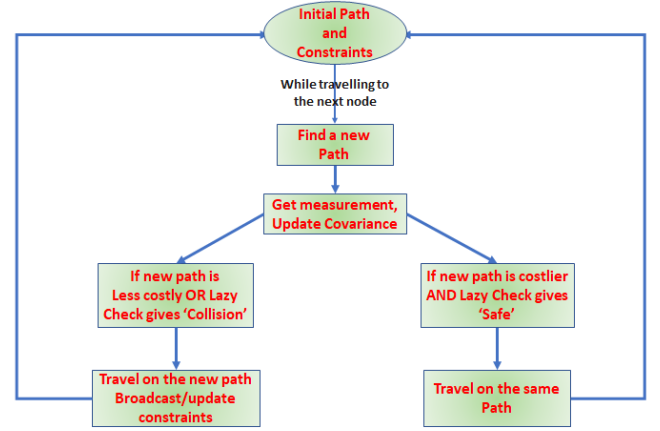


Figure 4. Execution Loop: Loop used to update agent trajectories.

5 Simulation Results

This section presents the simulation results¹ showing the effectiveness of the proposed algorithm. For all simulations, a square environment with sides of length 17 units is selected. A step size of 0.5 units was selected and the upper bound on the probability of collision M was selected as 0.2. As discussed in Section 2.2, the step size was selected smaller than the size of the static obstacles to avoid collision check for intermediate points on each edge. Both A and B matrices in the robot model (2), as well as the measurement matrix C_t were selected as the identity matrix. Each of the covariance matrices P_{x_0} , P_{ω} , and Q were selected as $\begin{bmatrix} 0.2 & 0 \\ 0 & 0.2 \end{bmatrix}$.

Figure 5 shows an illustration of the algorithm on a scenario comprised of four static obstacles, two robots, and a dynamic obstacle. The red robot has higher priority and decides its trajectory first. It can be seen that the CC-RRT for the blue robot yields a path around the trajectory of red robot and the dynamic obstacle.

Figure 6 shows illustration of the priority assignment feature of our algorithm on a scenario comprised of four static obstacles, two robots, and a dynamic obstacle. In this case, the green robot has higher priority. The path of blue robot crosses the path of both green robot and the magenta dynamic obstacle. However, the robots/dynamic obstacle are not present at the point of intersection at the same instance of time.

Finally, Figure 7 shows another illustration of the algorithm on a scenario comprised of five static obstacles, two robots, and a dynamic obstacle. The path of blue robot crosses the path of both green robot and the magenta dynamic obstacle. Here, the blue robot hovers around in zig-zag path and allows for the dynamic obstacle to pass before proceeding forward.

¹The code used to generate the simulation results is available at <https://github.com/yashbagla321/Multi-agent-CCRRT>

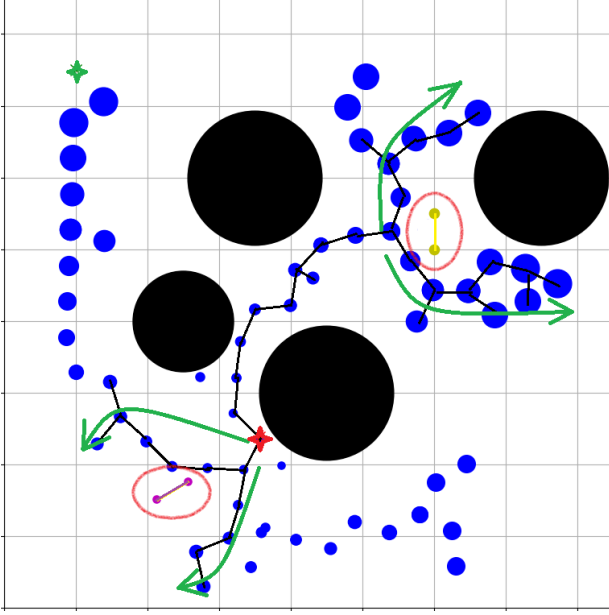


Figure 5. Obstacle avoidance: the tree is grown such that the robot avoids the path of the moving obstacle (yellow dots) and the other robot (magenta dots) while planning a path

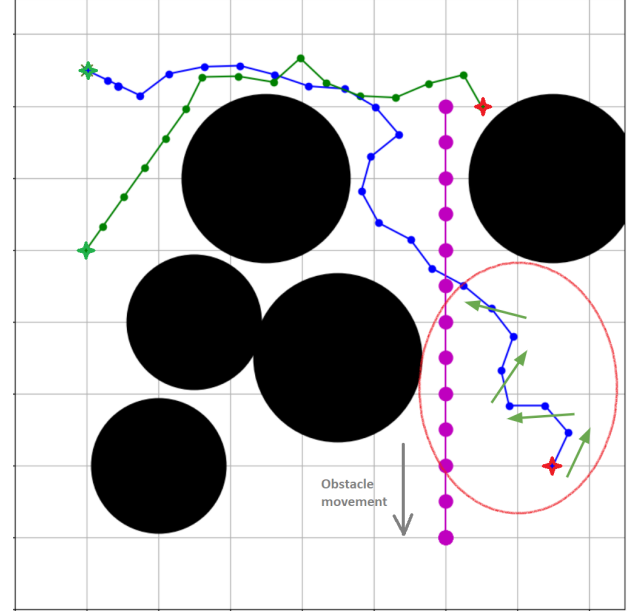


Figure 7. Scenario: The blue robot hovers around in a zig-zag path while the dynamic obstacle (magenta dots) passes and then proceeds towards its goal to avoid collision

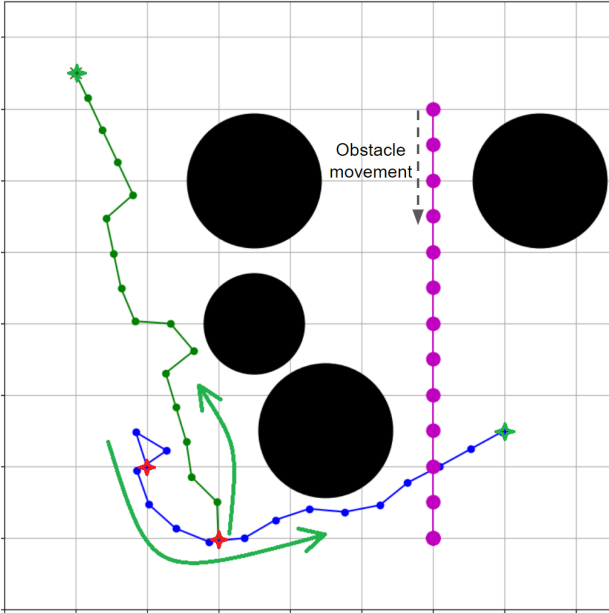


Figure 6. Priority Assignment: two robots blue and green navigating from a set of start nodes (red stars) to the set of goal nodes (green stars), in an environment with static (black) and dynamic (magenta) obstacles. The blue robot gives priority of way to green robot. Each dot represents one time step

6 Conclusions and Future Directions

In this paper, we studied an incremental motion planning algorithm for multiple robots with uncertain motion model that

are operating in an environment with static obstacles and uncertain dynamic obstacles. We adopted a sampling-based chance-constrained incremental motion planning algorithm, namely, the CC-RRT, as our base algorithm. We extended this algorithm to the case of priority ordered multiple robots. In particular, we let the robots with higher priority compute their trajectories first and then broadcast them for the lower priority robots. The low priority robots account for these trajectories while computing their trajectories. We implemented the algorithm in a receding horizon fashion in which each robot incorporates the measurement of its position at each time to improve the estimate of its position, and replans its trajectory based on the improved estimate. We illustrated our algorithms using several numerical examples.

The future directions for this work include the implementation of the algorithm with a biased sampling strategy that prefers to sample points in tight spaces and reduces the sample complexity of the algorithm. Another future direction is the design of alternative coordination strategies that would reduce the planning time by using an optimization technique to decide the priority order among all the robots. This will reduce the planning latency in the algorithm and would make it more amenable to a large number of robots.

REFERENCES

- [1] S. M. LaValle, *Planning Algorithms*. Cambridge University Press, 2006.
- [2] L. E. Kavraki, P. Svestka, J.-C. Latombe, and M. H. Over-

- mars, "Probabilistic roadmaps for path planning in high-dimensional configuration spaces," *IEEE Transactions on Robotics and Automation*, vol. 12, no. 4, 1996.
- [3] S. M. LaValle and J. J. Kuffner Jr, "Randomized kinodynamic planning," *The International Journal of Robotics Research*, vol. 20, no. 5, pp. 378–400, 2001.
 - [4] M. Elbanhawi and M. Simic, "Sampling-based robot motion planning: A review," *IEEE Access*, vol. 2, pp. 56–77, 2014.
 - [5] S. M. La Valle, "Motion planning," *IEEE Robotics & Automation Magazine*, vol. 18, no. 2, pp. 108–118, 2011.
 - [6] S. Karaman and E. Frazzoli, "Sampling-based algorithms for optimal motion planning," *The International Journal of Robotics Research*, vol. 30, no. 7, pp. 846–894, 2011.
 - [7] Y. Kuwata, J. Teo, G. Fiore, S. Karaman, E. Frazzoli, and J. P. How, "Real-time motion planning with applications to autonomous urban driving," *IEEE Transactions on Control Systems Technology*, vol. 17, no. 5, pp. 1105–1118, 2009.
 - [8] L. Ma, J. Xue, K. Kawabata, J. Zhu, C. Ma, and N. Zheng, "Efficient sampling-based motion planning for on-road autonomous driving," *IEEE Transactions on Intelligent Transportation Systems*, vol. 16, no. 4, pp. 1961–1976, 2015.
 - [9] S. Prentice and N. Roy, "The belief roadmap: Efficient planning in belief space by factoring the covariance," *The International Journal of Robotics Research*, vol. 28, no. 11–12, pp. 1448–1465, 2009.
 - [10] J. Van Den Berg, P. Abbeel, and K. Goldberg, "LQG-MP: Optimized path planning for robots with motion uncertainty and imperfect state information," *The International Journal of Robotics Research*, vol. 30, no. 7, pp. 895–913, 2011.
 - [11] A. Bry and N. Roy, "Rapidly-exploring random belief trees for motion planning under uncertainty," in *2011 IEEE International Conference on Robotics and Automation*, pp. 723–730, IEEE, 2011.
 - [12] A.-A. Agha-Mohammadi, S. Chakravorty, and N. M. Amato, "FIRM: Sampling-based feedback motion-planning under motion uncertainty and imperfect measurements," *The International Journal of Robotics Research*, vol. 33, no. 2, pp. 268–304, 2014.
 - [13] S. D. Bopardikar, B. Englot, A. Speranzon, and J. van den Berg, "Robust belief space planning under intermittent sensing via a maximum eigenvalue-based bound," *The International Journal of Robotics Research*, vol. 35, no. 13, pp. 1609–1626, 2016.
 - [14] B. Luders, M. Kothari, and J. How, "Chance constrained RRT for probabilistic robustness to environmental uncertainty," in *AIAA Guidance, Navigation, and Control Conference*, p. 8160, 2010.
 - [15] G. S. Aoude, B. D. Luders, J. M. Joseph, N. Roy, and J. P. How, "Probabilistically safe motion planning to avoid dynamic obstacles with uncertain motion patterns," *Autonomous Robots*, vol. 35, no. 1, pp. 51–76, 2013.
 - [16] L. Blackmore, H. Li, and B. Williams, "A probabilistic approach to optimal robust path planning with obstacles," in *2006 American Control Conference*, pp. 7–pp, IEEE, 2006.
 - [17] V. R. Desaraju and J. P. How, "Decentralized path planning for multi-agent teams with complex constraints," *Autonomous Robots*, vol. 32, no. 4, pp. 385–403, 2012.
 - [18] Y. Chen, M. Cutler, and J. P. How, "Decoupled multiagent path planning via incremental sequential convex programming," in *2015 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 5954–5961, IEEE, 2015.
 - [19] R. Kala, "Rapidly exploring random graphs: Motion planning of multiple mobile robots," *Advanced Robotics*, vol. 27, no. 14, pp. 1113–1122, 2013.
 - [20] W. Luo, N. Chakraborty, and K. Sycara, "Distributed dynamic priority assignment and motion planning for multiple mobile robots with kinodynamic constraints," in *2016 American Control Conference (ACC)*, pp. 148–154, IEEE, 2016.
 - [21] M. B. Dias, R. Zlot, N. Kalra, and A. Stentz, "Market-based multirobot coordination: A survey and analysis," *Proceedings of the IEEE*, vol. 94, no. 7, pp. 1257–1270, 2006.
 - [22] D. P. Bertsekas, "Auction algorithms for network flow problems: A tutorial introduction," *Computational Optimization and Applications*, vol. 1, no. 1, pp. 7–66, 1992.
 - [23] M. M. Zavlanos, L. Spesivtsev, and G. J. Pappas, "A distributed auction algorithm for the assignment problem," in *2008 47th IEEE Conference on Decision and Control*, pp. 1212–1217, IEEE, 2008.
 - [24] S. Omidshafiei, A.-A. Agha-Mohammadi, C. Amato, S.-Y. Liu, J. P. How, and J. Vian, "Decentralized control of multi-robot partially observable markov decision processes using belief space macro-actions," *The International Journal of Robotics Research*, vol. 36, no. 2, pp. 231–258, 2017.
 - [25] S. Thrun, W. Burgard, and D. Fox, *Probabilistic Robotics*. MIT press, 2005.
 - [26] S. S. Kia, S. Rounds, and S. Martinez, "Cooperative localization for mobile agents: A recursive decentralized algorithm based on kalman-filter decoupling," *IEEE Control Systems Magazine*, vol. 36, no. 2, pp. 86–101, 2016.
 - [27] S. D. Bopardikar, B. Englot, and A. Speranzon, "Multiobjective path planning: Localization constraints and collision probability," *IEEE Transactions on Robotics*, vol. 31, no. 3, pp. 562–577, 2015.
 - [28] J. Mattingley, Y. Wang, and S. Boyd, "Receding horizon control," *IEEE Control Systems Magazine*, vol. 31, no. 3, pp. 52–65, 2011.
 - [29] B. Siciliano, L. Sciavicco, L. Villani, and G. Oriolo, *Robotics: Modelling, Planning and Control*. Springer Science & Business Media, 2010.